国赛备战指南

祝贺各位进入国赛！以下是国赛备战的重点事项和技术要点。

1. 基础部分（按键、数码管、LED）

1.1 按键（KEY）模块

1.1.1 NE555 处理

当遇到 NE555 时，需要注意屏蔽 P34 引脚。在板子上可能不易察觉问题，直接删除 P34 相关代码即可：

```c
unsigned char Key_Read()
{
  unsigned char temp = 0;
  P44 = 0;  P42 = 1;  P35 = 1;
  if (P33 == 0)  temp = 4;
  if (P32 == 0)  temp = 5;
  if (P31 == 0)  temp = 6;
  if (P30 == 0)  temp = 7;

  P44 = 1;  P42 = 0;  P35 = 1;
  if (P33 == 0)  temp = 8;
  if (P32 == 0)  temp = 9;
  if (P31 == 0)  temp = 10;
  if (P30 == 0)  temp = 11;

  P44 = 1;  P42 = 1;  P35 = 0;
  if (P33 == 0)  temp = 12;
  if (P32 == 0)  temp = 13;
  if (P31 == 0)  temp = 14;
  if (P30 == 0)  temp = 15;
  return temp;
}
```

Fence 1

1.1.2 按键与串口共用引脚处理

按键和串口通信共用 P30 和 P31 引脚时，需遵循以下规则：

- **必须采用列扫描方式**：P30 和 P31 已被串口占用，矩阵键盘只能进行列扫描
- **禁止直接操作 P30/P31**：键盘扫描过程中不得对 P30 和 P31 赋值，以防串口发送错误数据
- **扫描前检测 P30 状态**：每次扫描前必须检测 P30 是否为高电平

关于 UART 通信中 RX 引脚电平变化：

- 空闲状态：RX 引脚处于**高电平**
- 接收数据时：电平根据数据内容动态变化
  - 起始位：低电平（逻辑 0）
  - 数据位：根据数据内容变化
  - 停止位：高电平（逻辑 1）

优化后的按键扫描代码，可删除不用的按键以降低误触发概率：

```c
#include "key.h"

unsigned char Key_Read()
{
    unsigned char temp = 0;
    if(P30 == 0) return 0;
    P44 = 0;P42 = 1;P35 = 1;P34 = 1;
    if(P30 == 0) temp = 7;
    if(P31 == 0) temp = 6;
    if(P32 == 0) temp = 5;
    if(P33 == 0) temp = 4;

    P44 = 1;P42 = 1;P35 = 1;P34 = 1;
    if(P30 == 0) return 0;
    P44 = 1;P42 = 0;P35 = 1;P34 = 1;
    if(P30 == 0) temp = 11;
    if(P31 == 0) temp = 10;
    if(P32 == 0) temp = 9;
    if(P33 == 0) temp = 8;

    P44 = 1;P42 = 1;P35 = 1;P34 = 1;
    if(P30 == 0) return 0;
    P44 = 1;P42 = 1;P35 = 0;P34 = 1;
    if(P30 == 0) temp = 15;
    if(P31 == 0) temp = 14;
    if(P32 == 0) temp = 13;
    if(P33 == 0) temp = 12;

    P44 = 1;P42 = 1;P35 = 1;P34 = 1;
    if(P30 == 0) return 0;
    P44 = 1;P42 = 1;P35 = 1;P34 = 0;
    if(P30 == 0) temp = 19;
    if(P31 == 0) temp = 18;
    if(P32 == 0) temp = 17;
    if(P33 == 0) temp = 16;

    P44 = 1;P42 = 1;P35 = 1;P34 = 1;
    P3 = P3|0xef;

    return temp;
}
```

Fence 2

1.1.3 双按键处理

对于双按键，必须放在最后判断，并使用 old 变量判断按下状态，避免单按键误触发：

```c
P44 = 0;P42 = 1;P35 = 1;P34 = 1;
if(P30 == 0) temp = 7;
if(P31 == 0) temp = 6;
if(P32 == 0) temp = 5;
if(P33 == 0) temp = 4;
if((P33 == 0) &&(P33 == 0))temp = 45;
```

Fence 3

1.2 数码管（SEG）模块

V4T 版本中无需维护 seg_point 数组，可以直接在 segproc 中维护所有数码管（Seg_Buf）：

```
Seg_Buf[0] = AD_in_100x / 100 % 10 + ',';
Seg_Buf[1] = AD_in_100x / 10 % 10;
Seg_Buf[2] = AD_in_100x % 10;
Seg_Buf[3] = 10;
Seg_Buf[4] = 10;
Seg_Buf[5] = 10;
Seg_Buf[6] = 10;
Seg_Buf[7] = 10;
```

Fence 4

中断处理中，通过判断 ASCII 值大小来确定是否带小数点（这里的判断只需要大于你的段选的长度就行了）：

```
// 数码管显示处理
 if (Seg_Buf[Seg_Pos] > 20)
   Seg_Disp(Seg_Pos, Seg_Buf[Seg_Pos] - ',', 1); // 带小数点
 else
   Seg_Disp(Seg_Pos, Seg_Buf[Seg_Pos], 0); // 无小数点
```

Fence 5

1.3 外设控制（LED、蜂鸣器、电机、继电器）

1.3.1 LED 闪烁实现

使用位变量和定时变量实现闪烁效果，主要是要注意一下定时器变量的类型，不要用错了：

```
bit led_flag;
unsigned char time_100ms;

ucLed[0]=led_flag;


if(++time_100ms==100)
{
    time_100ms=0;
    led_flag^=1;
}
```

Fence 6

1.3.2 LED 调光实现

通过 PWM 实现调光，在定时器中处理：

```
idata unsigned char pwm_period = 10;                                    // pwm调光周期，单位为毫秒
idata unsigned char pwm_compare = 5;                                    // pwm调光占空比，0-10可调，初始为5


 if (++pwm_period >= 10)
    pwm_period = 0;                 // pwm计数自增
  if (pwm_period < pwm_compare) // pwm计数比较
  {
    Led_Disp(ucLed); // LED显示更新
  }
  else // 熄灭led
  {
    Led_Off();
  }
```

Fence 7

1.3.3 蜂鸣器、继电器和电机控制

这三个外设需使用同一个全局变量：

```
idata unsigned char temp_0 = 0x00;
idata unsigned char temp_old_0 = 0xff;
/// @brief 蜂鸣器控制
/// @param enable
void Beep(bit enable)
{
  if (enable)
    temp_0 |= 0x40;
  else
    temp_0 &= ~(0x40);
  if (temp_0 != temp_old_0)
  {
    P0 = temp_0;
    P2 = P2 & 0x1f | 0xa0;
    P2 &= 0x1f;
    temp_old_0 = temp_0;
  }
}
/// @brief 继电器控制
/// @param enable
void Relay(bit enable)
{
  if (enable)
    temp_0 |= 0x10;
  else
    temp_0 &= ~(0x10);
  if (temp_0 != temp_old_0)
  {
    P0 = temp_0;
    P2 = P2 & 0x1f | 0xa0;
    P2 &= 0x1f;
    temp_old_0 = temp_0;
  }
}
/// @brief MOTOR控制
/// @param enable
void MOTOR(bit enable)
{
  if (enable)
    temp_0 |= 0x20;
  else
    temp_0 &= ~(0x20);
  if (temp_0 != temp_old_0)
  {
    P0 = temp_0;
    P2 = P2 & 0x1f | 0xa0;
    P2 &= 0x1f;
    temp_old_0 = temp_0;
  }
}
```

Fence 8

2. 常用外设（带参考代码）

2.1 IIC 总线设备（AD、DA、EEPROM）

2.1.1 AD 转换器

同时读取光敏电阻和滑动变阻器时会出现反向问题，以板子实际情况为准，交换读取地址可解决。

2.1.2 DA 输出

如果同时使用 AD 读取，AD读取应使用 0x4? 地址，避免输出断断续续。

2.1.3 EEPROM 读写

EEPROM 读取操作应放在定时器中断声明之前，或在读取过程中关闭/开启中断：

```
System_Init();    // 系统初始化
EEPROM_Read(Eep_Read, 0, 8);   // 读取1-8覆盖全为0的数组
Uart1_Init();     // 串口1初始化
```

Fence 9

或直接在底层函数中控制中断：

```
void EEPROM_Read(unsigned char *str, unsigned char addr, unsigned char num)
{
    I2CStart();
    I2CSendByte(0xa0);
    I2CWaitAck();
    I2CSendByte(addr);
    I2CWaitAck();


    I2CStart();
    I2CSendByte(0xa1);
    I2CWaitAck();
    EA=0;
    while (num--)
    {
        *str++ = I2CReceiveByte();
        if (num)
            I2CSendAck(0);
        else
            I2CSendAck(1);
    }
    I2CStop();
    EA=1;
}
```

Fence 10

写入的时候我们多加几个delay，因为连续写入的时候需要保证时序，并且我们需要对数据进行加锁，这里的iicdelay你可以加10个都行，主要是要保证你的全部写入没关系，加锁这个我在之前也是一直在提

```
void EEPROM_Write(unsigned char *str, unsigned char addr, unsigned char num)
{
  I2CStart();
  I2CSendByte(0xa0);
  I2CWaitAck();
  I2CSendByte(addr);
  I2CWaitAck();

  while (num--)
  {
    I2CSendByte(*str++);
    I2CWaitAck();
    I2C_Delay(200);
  }
  I2CStop();
  I2C_Delay(255);
  I2C_Delay(255);
  I2C_Delay(255);
  I2C_Delay(255);
  I2C_Delay(255);
  I2C_Delay(255);
}
```

2.2 DS1302 实时时钟

建议使用十进制处理，更加直观。写入时需先停止时钟芯片的 CH 位，避免时间跳变问题（1:20:59写入，概率变为1:21:59）。读取时记得关闭/开启中断：

```
void Set_Rtc(unsigned char *ucRtc)
{
  unsigned char i;
  Write_Ds1302_Byte(0x8e, 0x00); // 禁止写保护
  Write_Ds1302_Byte(0x80, 0x80); // 停止时钟芯片
  for (i = 0; i < 3; i++)
    Write_Ds1302_Byte(0x84 - 2 * i, ucRtc[i] / 10 % 10 * 16 + ucRtc[i] % 10);
  Write_Ds1302_Byte(0x8e, 0x80);
}

void Read_Rtc(unsigned char *ucRtc)
{
  unsigned char i;
  unsigned temp;
  EA = 0; // 关闭总中断，防止时序错误
  for (i = 0; i < 3; i++)
  {
    temp = Read_Ds1302_Byte(0x85 - 2 * i);
    ucRtc[i] = temp / 16 * 10 + temp % 16;
  }
  EA = 1;
}
```

Fence 12

2.3 DS18B20 温度传感器

温度读取时间应根据题目要求设置，建议略小于要求值（如要求 200ms 则设置 180ms），不要过快读取，以确保时序正确。

3. 特殊外设（无参考代码）

3.1 超声波模块

若使用 Y1 指令集导致测量范围限制在 7cm，可以对准地面测量或更换为 Y5 指令集。

3.2 串口通信

确认波特率正确（检查 ISP 和 UART 初始化）后，不要使用判断条件，直接回显接收到的信息进行验证。建议使用超时解析加重定向方式处理串口数据。

注意：如不使用串口中断，请勿开启 ES 中断使能位，避免 printf 函数卡死：

```
/* 串口数据 */
pdata unsigned char Uart_Buf[10] = {0}; // 串口接收缓冲区
idata unsigned char Uart_Rx_Index;      // 串口接收索引
idata unsigned char Uart_Recv_Tick;     // 串口接收时间标志
idata unsigned char Uart_Rx_Flag;


void Uart_Proc()
{

  if (Uart_Rx_Index == 0)
    return; // 无数据，直接返回

  if (Uart_Recv_Tick >= 10)
```

国赛用单片机 - 米醋工作室_左岚

```
  { // 若接收超时，对数据读取，清空缓冲区
    Uart_Recv_Tick = 0;
    Uart_Rx_Flag = 0;
    printf("%s", Uart_Buf);
    memset(Uart_Buf, 0, Uart_Rx_Index); // 清空接收数据
    Uart_Rx_Index = 0;
  }
}

void Uart1_Isr(void) interrupt 4
{
  // 若接收到数据
  if (RI)
  {
    Uart_Rx_Flag = 1;                  // 接收标志
    Uart_Recv_Tick = 0;                // 清零接收时间标志
    Uart_Buf[Uart_Rx_Index++] = SBUF; // 将数据保存到缓冲区
    RI = 0;                            // 清除接收中断标志
    if (Uart_Rx_Index > 10)
    {
      Uart_Rx_Index = 0; // 防止溢出
      memset(Uart_Buf, 0, 10);
    }
  }
}
```

Fence 13

4. 问题排查方案

4.1 无串口通信情况

如果题目不涉及串口，可以通过按键控制测试其他模块功能。国赛必备调试技巧，详见单片机调试讲解。

注意：使用调试功能时，顶部两行按键无法使用，需相应调整测试方案。

4.2 有串口通信情况

先确保其他模块正常工作，串口回显无误后，逐步叠加测试串口功能。例如，可通过串口控制点亮 LED 来验证功能。

5. 编程注意事项与常见错误

5.1 常见错误

1. **按键底层变量未初始化**：Key_Read 函数中的 temp 变量未初始化为 0

2. **串口中断错误**：开启串口中断但未编写中断处理函数，导致系统卡死

5.2 变量与内存管理

1. **数据类型转换**：float 运算后进行显式类型转换，特别是乘以 100 后转为 unsigned char 时容易溢出

2. **数组边界检查**：防止数组索引越界，确保索引在有效范围内

3. **内存区域初始化**：使用 xdata 或 pdata 存储区时必须显式初始化变量

4. **局部变量初始化**：按键处理函数中的临时变量必须初始化为 0

5. **类型转换:**务必在公式计算的时候，使用float将所有数据转换为float型，还有就是常数使用1.0f这种形式转换

5.3 显示处理

1. **高位数字熄灭**：禁止使用 while 循环进行高位零熄灭，应使用三目运算符

2. **防止闪烁**：避免对同一变量进行两次赋值，防止中断导致显示闪烁

5.4 初始化

我们的定时器0（频率测量）要在定时器1前初始化，调度器要在定时器0后初始化，EEPROM的读取要在所有初始化之前

```
System_Init();              // 系统初始化
rd_temperature();           // 读取温度
EEPROM_Read(Eep_Read, 0, 8); // 测试eeprom
Uart1_Init();               // 串口1初始化
Set_Rtc(ucRtc);             // 设置时钟
Timer0_Init();              // 定时器0初始化
Scheduler_Init();           // 调度器初始化
Timer1_Init();              // 定时器1初始化
```

Fence 14